Choreographer

System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-Choreographer-go.

Choreographer - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/Choreographer. Entry point: ./cmd/choreographer. Module: arrowhead/ah5-support-go/choreographer. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8457 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8457"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Choreographer	Plan DAG validation, executor registration, synchronous orchestration, persistent steps/callbacks/worklog	Async result callback (explicit 501), complete dependency negotiation, optimal executor selection, session notification, remote abort propagation, inter-cloud sessions

Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-Choreographer-rust.

Choreographer (Rust)

Arrowhead 5.2.2 Choreographer support system. Entry point src/main.rs is built as binary choreographer by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/choreographer.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/Choreographer, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin choreographer -- <config.json>

Listening port

Default 127.0.0.1:8457 (Arrowhead default, role choreographer). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Choreographer	Plan/executor CRUD, single local action/step starts, aborts and completion callbacks	Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native

snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-Choreographer-go/documentation/Choreographer/Choreographer-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Choreographer/Choreographer-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

Choreographer

System Design Description

Abstract

This document describes the design and implementation of the Choreographer support system of Arrowhead 5.2.2 (`ah5-Choreographer-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Independent source and build ownership 4
2.3 Functional properties implementation 4
2.4 Non functional properties implementation 5
2.5 Deployment 5
2.6 Local modifications 6
2.7 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the Choreographer system implementation, which realizes the Choreographer SysD [1]. The Choreographer executes workflow plans: for each step it selects an executor system, has Dynamic ServiceOrchestration orchestrate the required provider for the executor, starts the executor and follows the results. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation is based on the SysD document Choreographer [1] and is located in `ah5-Choreographer-java-spring`.

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9, Arrowhead common library 5.2.2 (private source under `vendor/`), Log4j 2; MariaDB Connector/J against MySQL 8.4 with TLS in the standalone deployment.
- Database `ah_choreographer` on MySQL 8.4 (MariaDB Connector/J, TLS with server verification in the standalone deployment), schema validated by Hibernate; SQL scripts in `src/main/resources/database`. Stateless request processing; scheduled jobs use Quartz.

2.2 Independent source and build ownership

The project folder owns its application source, Maven build, tests, executable JAR, Dockerfile, configuration and Compose definition. No source or image is taken from a sibling support folder or JavaSupportRuntime. Build with Java 21 and `mvn -B clean verify`; a populated external Maven dependency cache permits offline builds.

Arrowhead common utilities and DTOs are private source copies in `vendor/`, compiled into this system by the local Maven POM. No Arrowhead common/core parent or separately installed Arrowhead library is needed. Upstream POMs, licenses and source hashes are retained for provenance. Security updates must be propagated and tested in every affected copy. Third-party dependencies remain ordinary Maven libraries.

The packaged artifact is `arrowhead-choreographer-5.2.2.jar`. The image builds and tests this folder only; the system has a fixed application main class.

2.3 Functional properties implementation

The following components implement the system:

Component	Responsibility
`api.http`	Controllers of the four services and the orchestration-result endpoint.
`engine`	Session engine and step graph.
`service`	Plans, executors, sessions.
`jpa`	Entities.

Every request passes the common filter chain: authentication filter (`authentication.policy`), optional blacklist filter, management filter (management services), the system's authorization filter where present, then the controller, validation and normalization, the service logic and the database service.

2.3.1 Data handled and stored

Storage `ah_choreographer`:

Tables	Content
`plan, plan_action, plan_step, plan_step_connection`	Plans.
`executor, executor_service_definition`	Executors.

Tables	Content
`plan_session, session_step, worklog`	Execution.
`logs`	Log entries.

The data models of the interfaces are Java records (of `eu.arrowhead.dto` in the common library, or of the module); the IDD documents name them without the `DTO` suffix. The records used by this system are: `ActionRequestDTO`, `ActionResponseDTO`, `CheckPlanResponseDTO`, `ExecutedStepResultDTO`, `ExecutorCreateRequestDTO`, `ExecutorListResponseDTO`, `ExecutorRegistrationRequestDTO`, `ExecutorResponseDTO`, `ExecutorServiceDefinitionResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `OrchestrationServiceRequirementDTO`, `PageDTO`, `PlanListResponseDTO`, `PlanRequestDTO`, `PlanResponseDTO`, `RunPlanListRequestDTO`, `RunPlanListResponseDTO`, `RunPlanRequestDTO`, `RunPlanResponseDTO`, `SessionListResponseDTO`, `SessionResponseDTO`, `SessionStepListResponseDTO`, `SessionStepResponseDTO`, `StartSessionListRequestDTO`, `StartSessionRequestDTO`, `StepRequestDTO`, `StepResponseDTO`, `WorklogListResponseDTO`, `WorklogResponseDTO`.

2.4 Non functional properties implementation

2.4.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore, `client-auth=need`). The common library's filters implement the authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking.

2.4.2 Power management

None; the system is a server process without power states. The `sysMonitor power` observation always reports line supply.

2.4.3 Internal monitoring

Log4j 2 logs to stdout and files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The common library's `SysMonitorPublisher` publishes `echo`, `power` and `status` every 30 s.

2.4.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD; `generalManagement.get-config` returns the public ones.

2.5 Deployment

The local `compose.yaml` selects only this support system and its private MySQL database. It joins an operator-selected external cloud network; it does not start core systems or another support system. Copy `.env.example`, provision this system's own identity and credential files, validate the Compose definition, then run `docker compose up -d --build` from this folder. The image uses the secure external `config/application.properties`, mutual TLS, outsourced authentication, certificate/token binding and a mounted CRL. Secrets are read through Spring's file configuration tree. The app runs as UID 10001 with a read-only root filesystem and bounded non-executable temporary/log storage.

The database image initializes this system's own schema on an empty project-owned volume, reads independent random user/root passwords from files and uses a dedicated internal network. It has no host port. The application verifies the database TLS certificate against its public DB CA. The DB key and CA signing

key are never mounted into the application. Distinct Compose project names, host ports and private subnets are required for multiple deployments. Existing volumes require explicit migration/backup handling; ordinary shutdown preserves them.

Registry, Authentication, DSO, ConsumerAuthorization and required brokers/providers remain network dependencies. Independent deployment does not remove those service contracts. The listener health check is only a liveness check; inspect successful application startup, registration and authenticated service access before acceptance. The local `DEPLOYMENT.md` supplies the complete procedure. The previously recorded live-cloud deployment is historical baseline evidence; the new packaging does not change a live cloud.

2.6 Local modifications

Notes on this implementation in the workspace:

- Migration of v4.6; DSO push orchestration replaces orchestration by proxy.

2.7 Verification

Independent folder-only Maven builds run the project's JUnit suite. Artifact inspection checks the system main/source and absence of JavaSupportRuntime or other support-system implementations. Images and project-local Compose definitions are tested with private disposable state and generated test identities. Spring startup and authenticated HTTPS requests use isolated mutual-TLS core-service fixtures; missing-token and missing-certificate requests are refused. The distribution verification report records exact results and limitations. Broker, NTP and inter-cloud acceptance remain separate from deployment/startup validation.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`choreographerSession`	[1]	[2]	[3]
`choreographerExecutor`	[1]	[4]	[5]
`choreographerManagement`	[1]	[6]	[7]
`generalManagement`	[1]	[8]	[9]
`sysMonitor`	[1]	[10]	[11]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[12]	[13]	–
identity	[14]	[15]	–
authorization	[16]	[17]	–
serviceOrchestrationPush Management	[18]	[19]	–
executor interface	Executors	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

4 References

[1] J. Delsing, “Choreographer – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/Choreography/documentation/v5.2.2/Choreographer-v5.2.2-SysD.pdf.

[2] ——, “choreographerSession – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerSession-SD.pdf.

[3] ——, “choreographerSession HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerSession-HTTP-IDD.pdf.

[4] ——, “choreographerExecutor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerExecutor-SD.pdf.

[5] ——, “choreographerExecutor HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerExecutor-HTTP-IDD.pdf.

[6] ——, “choreographerManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerManagement-SD.pdf.

[7] ——, “choreographerManagement HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-choreographerManagement-HTTP-IDD.pdf.

[8] ——, “generalManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-generalManagement-SD.pdf.

[9] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-generalManagement-HTTP-IDD.pdf.

[10] ——, “sysMonitor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-sysMonitor-SD.pdf.

[11] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Choreographer),” Eclipse Arrowhead, IDD 5.2.2, 2026, choreographer-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[12] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[13] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[14] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[15] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[16] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[17] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-SD.pdf.

[18] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[19] ——, “serviceOrchestrationPushManagement – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Self-contained build, private library source, configuration, images, database and isolated deployment verification.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Choreographer

1.1 System overview

Plan, executor and dependency-ordered session management. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

1.2 Produced and consumed services

Produced HTTPS services: `choreographerExecutor`, `choreographerManagement`, `choreographerSession`, `echo`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Local synchronous orchestration and executor start/callback flow are implemented. Async orchestration callbacks return 501. Java executor dependency negotiation, optimal selection, session notifications and remote abort propagation are incomplete.

1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

2 Implementation

Go 1.26 or newer builds the native binary. State is serialized as private JSON documents; updates clone state, validate, persist and then publish. Server timeouts and bounded message sizes constrain resource use. Persistent outboxes are at-least-once and cannot provide distributed exactly-once delivery. Detailed source, configuration and test instructions are in the repository README and verification report.

2.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/executors	management
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/session/start	identity
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/session/abort/{id}	identity
DELETE	/choreographer/mgmt/sessions/{id}	management
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/orchestration-result/{correlationId}	identity
GET	/choreographer/monitor/echo	certificate
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management

1 Choreographer

1.1 System overview

Native Rust development profile for local Java source `Choreography/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Plan/executor CRUD, single local action/step starts, aborts and completion callbacks

1.5 Open release gates

Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `choreographerExecutor`, `choreographerManagement`, `choreographerSession`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management
GET	/choreographer/monitor/echo	certificate
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/session/start	identity
DELETE	/choreographer/session/abort/{id}	identity
POST	/choreographer/orchestration-result/{correlationId}	identity / 501
GET	/choreographer/mgmt/plans	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/mgmt/sessions/{id}	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
POST	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.